

## Lab 1:

The screenshot displays two side-by-side instances of the pgAdmin 4 web interface, both connected to a PostgreSQL database named 'dvdrental'.

**Left Window:**

- Query Editor:** Contains a SQL query labeled '-- Step 1'. The query is:

```
BEGIN ;  
UPDATE film SET rental_rate = 4.99  
WHERE film_id = 1 ;  
-- Holds X-Lock on film_id = 1  
  
COMMIT;
```
- Data Output / Messages:** Shows the message 'UPDATE 1' followed by 'Query returned successfully in 79 msec.'
- Status Bar:** Displays 'Total rows: Query complete 00:00:00.079 CRLF Ln 2, Col 1'.

**Right Window:**

- Query Editor:** Contains a SQL query:

```
BEGIN ;  
UPDATE film SET rental_rate = 5.99  
WHERE film_id = 1 ;  
  
COMMIT;
```
- Data Output / Messages:** Shows the message 'COMMIT' followed by 'Query returned successfully in 78 msec.'
- Status Bar:** Displays 'Total rows: Query complete 00:00:00.078 CRLF Ln 5, Col 1'. A green notification bubble above the status bar states '✓ Query returned successfully in 79 msec.'.

Both windows have tabs for 'Query', 'Query History', and 'Scratch Pad'. The bottom of each window has tabs for 'Data Output', 'Messages', and 'Notifications'.

The image shows two side-by-side PostgreSQL query editors. The left editor contains a SQL script for 'Step 1' that begins a transaction, updates the rental\_rate of film\_id 1 to 4.99, holds an X-lock, and then commits. The right editor contains a script that begins a transaction, updates the rental\_rate of film\_id 1 to 5.99, and then commits. Both editors show a successful query completion message in the Data Output pane.

```
-- Step 1
BEGIN ;
UPDATE film SET rental_rate = 4.99
WHERE film_id = 1 ;
-- Holds X-Lock on film_id = 1

COMMIT;
```

```
BEGIN ;
UPDATE film SET rental_rate = 5.99
WHERE film_id = 1 ;

COMMIT;
```

COMMIT  
Query returned successfully in 107 msec.

COMMIT  
Query returned successfully in 124 msec.

✓ Query returned successfully in 124 msec. ✕

The result shows that Session B is blocked while attempting to update a row locked by Session A. After Session A commits, Session B successfully executes its update, and the final `rental_rate` becomes `5.99`. This confirms that PostgreSQL prevents conflicting updates until the existing lock is released.

## Lab 2:

The image shows two side-by-side PostgreSQL query editors. The left editor contains a script for 'Step 1' (a transaction with a select) and 'Step 3' (another transaction with a select). The right editor contains a script that begins a transaction, updates the rental\_rate of film\_id 1 to 0.99, and then rolls back. The left editor shows the result of the select query in the Data Output pane, displaying a table with rental\_rate 4.99. The right editor shows a successful rollback message.

```
-- Step 1
BEGIN ;
SELECT rental_rate FROM film
WHERE film_id = 10;

-- Step 3
SELECT rental_rate FROM film
WHERE film_id = 10;
NEW RESULT : 0.99!
COMMIT;
```

```
BEGIN ;
UPDATE film SET rental_rate = 0.99
WHERE film_id = 1 0;

COMMIT;
```

rollback

Query returned successfully in 96 msec.

✓ Successfully run. Total query runtime: 136 msec. 1 rows affected. ✕

The screenshot shows a PostgreSQL client window with the following SQL query in the editor:

```

1 -- Step 1
2 BEGIN ;
3 SELECT rental_rate FROM film
4 WHERE film_id = 10;
5
6 -- Step 3
7 SELECT rental_rate FROM film
8 WHERE film_id = 10;
9 NEW RESULT : 0.99!
10 COMMIT;
11

```

The Data Output tab shows the result of the first SELECT statement:

| rental_rate |
|-------------|
| 4.99        |

The status bar indicates: Total rows: 1, Query complete 00:00:00.136, CRLF, Ln 2, Col 1.

The screenshot shows a PostgreSQL client window with the following SQL query in the editor:

```

1 -- Step 1
2 BEGIN ;
3 SELECT rental_rate FROM film
4 WHERE film_id = 10;
5
6 -- Step 3
7 SELECT rental_rate FROM film
8 WHERE film_id = 10;
9 COMMIT;
10

```

The Data Output tab shows the result of the first SELECT statement:

| rental_rate |
|-------------|
| 0.99        |

The status bar indicates: Total rows: 1, Query complete 00:00:00.140, CRLF, Ln 10, Col 1.

The result shows that the same **SELECT** statement returns different values within a single transaction because another transaction updates and commits the data between the two reads. This confirms that under **READ COMMITTED**, each query sees the latest committed data rather than maintaining a consistent snapshot throughout the transaction.

### Lab 3:

dvdtental/postgres@PostgreSQL 18

Query

Query History

Scratch Pad

1

BEGIN;

2

UPDATE film SET rental\_rate =2.99

3

WHERE film\_id = 100;

Data Output

Messages

Notifications

UPDATE 1

Query returned successfully in 88 msec.

dvdtental/postgres@PostgreSQL 18

Query

Query History

Scratch Pad

1

BEGIN;

2

UPDATE film SET rental\_rate =2.99

3

WHERE film\_id =200;

Data Output

Messages

Notifications

UPDATE 1

Query returned successfully in 88 msec.

dvdtental/postgres@PostgreSQL 18

Query

Query History

Scratch Pad

1

BEGIN;

2

UPDATE film SET rental\_rate =2.99

3

WHERE film\_id = 100;

Data Output

Messages

Notifications

UPDATE 1

Query returned successfully in 72 msec.

dvdtental/postgres@PostgreSQL 18

Query

Query History

Scratch Pad

1

BEGIN;

2

UPDATE film SET rental\_rate =2.99

3

WHERE film\_id =200;

Data Output

Messages

Notifications

UPDATE 1

Query returned successfully in 78 msec.

The screenshot shows two instances of pgAdmin. The left instance shows a query window with the following SQL:

```

1 BEGIN;
2 UPDATE film SET rental_rate =2.99
3 WHERE film_id = 100;
4 UPDATE film SET rental_rate =3.99
5 WHERE film_id =200;

```

The Data Output tab shows: "UPDATE 1. Query returned successfully in 1 min 46 secs."

The right instance shows a query window with the following SQL:

```

1 BEGIN;
2 UPDATE film SET rental_rate =2.99
3 WHERE film_id =200;
4 UPDATE film SET rental_rate = 3.99
5 WHERE film_id = 100;

```

The Data Output tab shows a deadlock error:

```

ERROR: deadlock detected
Process 5528 waits for ShareLock on transaction 1016; blocked by process 26104.
Process 26104 waits for ShareLock on transaction 1017; blocked by process 5528.

SQL state: 40P01
Detail: Process 5528 waits for ShareLock on transaction 1016; blocked by process 26104.
Process 26104 waits for ShareLock on transaction 1017; blocked by process 5528.
Hint: See server log for query details.
Context: while locking tuple (5,12) in relation "film"

```

The result shows that both sessions become blocked when attempting to access rows locked by each other. PostgreSQL detects the deadlock and aborts one transaction with a deadlock error, allowing the other transaction to continue and complete successfully. This demonstrates how PostgreSQL automatically resolves deadlocks to prevent indefinite waiting.

## Lab 4:

The screenshot shows two instances of pgAdmin. The left instance shows a query window with the following SQL:

```

1 BEGIN ;
2 SELECT *
3 FROM inventory
4 WHERE inventory_id = 10 FOR UPDATE ;
5
6 INSERT INTO rental ( rental_date , inventory_id
7 VALUES (NOW( ) , 10 , 1 , 1 ) ;
8 COMMIT;

```

The Data Output tab shows a table with the following data:

| inventory_id [PK] integer | film_id smallint | store_id smallint | last_update timestamp without time zone | currently_rented boolean |
|---------------------------|------------------|-------------------|-----------------------------------------|--------------------------|
| 1                         | 10               | 2                 | 2006-02-15 10:09:17                     | false                    |

The right instance shows a query window with the following SQL:

```

1 BEGIN ;
2 SELECT *
3 FROM inventory
4 WHERE inventory_id = 10 FOR UPDATE ;
5
6 10 ROLLBACK ;

```

The Data Output tab shows: "ROLLBACK. Query returned successfully in 66 msec."

At the bottom of the right window, a green status bar indicates: "Successfully run. Total query runtime: 196 msec. 1 rows affected."

The image displays two screenshots of a PostgreSQL IDE interface, likely pgAdmin, showing the execution of SQL queries and the resulting data output.

**Top Screenshot:** The left pane shows a query window with the following SQL code:

```
1 BEGIN ;
2 SELECT*
3 FROM inventory
4 WHERE inventory_id = 10 FOR UPDATE ;
5
6 INSERT INTO rental ( rental_date , inventory_id
7 VALUES (NOW( ) , 10 , 1 , 1 ) ;
8 COMMIT;
```

The right pane shows the query history and the data output. The data output is a table with the following columns: `inventory_id` (integer), `film_id` (smallint), `store_id` (smallint), `last_update` (timestamp without time zone), and `currently_rented` (boolean). The table contains one row:

| inventory_id | film_id | store_id | last_update                | currently_rented |
|--------------|---------|----------|----------------------------|------------------|
| 10           | 2       | 2        | 2026-09-25 15:00:20.532862 | true             |

The status bar indicates "Query complete 00:00:00.162".

**Bottom Screenshot:** The left pane shows the same SQL code as the top screenshot. The right pane shows the query history and the data output. The data output is a table with the following columns: `inventory_id` (integer), `film_id` (smallint), `store_id` (smallint), `last_update` (timestamp without time zone), and `currently_rented` (boolean). The table contains one row:

| inventory_id | film_id | store_id | last_update                | currently_rented |
|--------------|---------|----------|----------------------------|------------------|
| 10           | 2       | 2        | 2026-09-25 15:00:20.532862 | true             |

The status bar indicates "Query complete 00:00:00.096". A green message box at the bottom right states: "Query returned successfully in 96 msec."

The result shows that **SELECT FOR UPDATE** locks the selected row, causing the other session to wait until the first transaction completes. After the lock is released, the waiting transaction can proceed using the updated data. This confirms that row-level locking helps prevent conflicting updates and maintains data consistency during concurrent transactions.